JADAVPUR UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Assignment 1 Report – Comprehensive Evaluation of Error Detection Schemes in Network Transmission

Error detection with 16-Bit Internet Checksum and Cyclic Redundancy Checks (CRC-8, 10, 16, 32)

Course: Computer Networks Laboratory (BCSE-III)
Student Name: Sampad De
Section: A1
Roll Number: 002410501025
Assignment: Assignment 1 (Error Detection)
Date of Submission: August 21, 2026

Contents

Contents 1

1 Introduction and Objectives 3

2 Theoretical Principles of Error Detection 3
2.1 The 16-Bit Internet Checksum (One's Complement Sum) 3
2.1.1 Theoretical Vulnerabilities of the Checksum 4
2.2 Cyclic Redundancy Check (CRC) and Modulo-2 Arithmetic 4
2.2.1 Mathematical Guarantees of CRC Polynomials 5

3 System Architecture and Implementation 5
3.1 Frame Structure and In-Memory Layout 5
3.2 Development Setup and Evaluation Environment 6
3.3 Implementation of Checksum and CRC Algorithms 7
3.4 CRC Polynomial Constants and Specifications 8

4 Methodology and Error Injection Models 9
4.1 Client-Server UDP Pipeline 9
4.2 Error Injection Engine and Resolution of Edge-Case Bugs 10
4.3 Burst Error Modeling 11

5 Experimental Results and Empirical Evaluation 11
5.1 Single Bit Error Results 11
5.2 Isolated Double Bit Error Results 12
5.3 Odd Number of Errors Results 12
5.4 Uniform Burst Error Results (Lengths 12 and 32) 13
5.4.1 Theoretical vs. Empirical Burst Bounds 13

6 Computational Efficiency and Benchmarks 14

7 Concrete Boundary Collisions (Hex Dumps) 15
7.1 Case 1: Error Detected by Both Schemes 15
7.2 Case 2: Detected by Checksum, Missed by CRC-8 (CRC Collision) . . . 15
7.3 Case 3: Detected by CRC-8, Missed by Checksum (Compensating Sum) . 15

8 Testing Conditions, Limitations, and Enhancements 16
8.1 Ideal Testing Conditions vs. Real-World Environments 16
8.2 Known Limitations and Practical Improvements 16

9 Conclusion **17**

10 Sources Used **17**

System Configuration and Environment

This project was developed and evaluated using the following environment:

Component	Specification
Operating System	macOS
Networking	BSD Sockets (UDP)
Compiler	Native Clang (`clang`)
Profiling Timer	POSIX `clock_gettime(CLOCK_MONOTONIC)`

1 Introduction and Objectives

Data transmitted across any physical communications medium is subject to corruption caused by thermal noise, electrical interference, line attenuation, and clock synchronization drift. In computer network protocol stacks, ensuring that received bits match transmitted bits is divided between the Data Link Layer and Transport Layer.

For this assignment, I designed and implemented a modular network simulation environment in C. The goals of my project are:

1. **Protocol Implementation:** Implement the standard 16-bit Internet Checksum (RFC 1071) and four variations of Cyclic Redundancy Checks (CRC-8, CRC-10, CRC-16, and CRC-32) from scratch.

2. **Client-Server Architecture:** Construct a socket-based sender and receiver application running over UDP to transmit framed text payloads across a network interface.

3. **Deterministic and Statistical Error Injection:** Develop an error-injection engine capable of inserting single-bit flips, isolated double-bit flips, odd-parity bit flips, and uniformly distributed burst errors of varying lengths.

4. **Empirical and Theoretical Validation:** Run a large-scale offline benchmarking suite (1,000,000 simulated frame transmissions) to measure exact detection rates, identify edge-case collisions, and quantify computational throughput on macOS.

2 Theoretical Principles of Error Detection

2.1 The 16-Bit Internet Checksum (One's Complement Sum)

The Internet Checksum is an arithmetic error-detection algorithm used in IPv4, TCP, and UDP headers. The algorithm treats the input byte stream as a sequence of 16-bit unsigned integers.

The arithmetic uses **one's complement addition**. In standard two's complement computer hardware, when the sum of two 16-bit integers produces a carry out of the 16th bit (bit 15), this overflow bit represents $2^{16} \equiv 1 \pmod{2^{16}-1}$. Therefore, an **end-around carry** is performed: the carry bit is added back into the least significant bit of the sum. Finally, the total sum is bitwise inverted (one's complemented).

$$\text{Checksum} = \sim \left(\sum_{i=1}^{k} W_i \pmod{2^{16}-1} \right) \tag{1}$$

2.1.1 Theoretical Vulnerabilities of the Checksum

While simple to compute in software and hardware, the checksum has well-known structural weaknesses:

- **Compensating Errors:** If an error in word W_a increments its value by $+K$ and another error in word W_b decrements its value by $-K$, the total sum remains completely identical.
- **Byte Order Invariance:** Swapping two 16-bit words or rearranging whole aligned blocks produces the exact same sum.
- **Zero Preservation:** A block of all-zero words sums to zero and does not change the checksum.

2.2 Cyclic Redundancy Check (CRC) and Modulo-2 Arithmetic

Cyclic Redundancy Checks treat binary bit strings as coefficients of polynomials using modulo-2 arithmetic. In modulo-2 binary arithmetic, addition and subtraction are equivalent and correspond directly to the bitwise Exclusive-OR (XOR) operation without carries or borrows.

Given an m-bit message polynomial $M(x)$ and an $(r+1)$-bit generator polynomial $G(x)$ of degree r:

1. The message is multiplied by x^r, effectively appending r zero bits: $x^r M(x)$.
2. I performed modulo-2 polynomial division of $x^r M(x)$ by $G(x)$ to obtain a quotient $Q(x)$ and an r-bit remainder $R(x)$:

$$x^r M(x) = Q(x)G(x) \oplus R(x) \tag{2}$$

3. The transmitted frame polynomial $T(x)$ is formed by appending the remainder $R(x)$ to the message:

$$T(x) = x^r M(x) \oplus R(x) = Q(x)G(x) \tag{3}$$

Since $R(x) \oplus R(x) = 0$ in modulo-2 arithmetic, $T(x)$ is an exact multiple of $G(x)$.

When the receiver gets a potentially corrupted frame $T'(x) = T(x) \oplus E(x)$, where $E(x)$ is the error polynomial, it computes the remainder of $T'(x)/G(x)$:

$$\frac{T'(x)}{G(x)} = \frac{T(x) \oplus E(x)}{G(x)} = Q(x) \oplus \frac{E(x)}{G(x)} \tag{4}$$

An error is undetected **if and only if** the error polynomial $E(x)$ is evenly divisible by $G(x)$.

2.2.1 Mathematical Guarantees of CRC Polynomials

- **Single-Bit Errors ($E(x) = x^i$):** If $G(x)$ has at least two non-zero terms (e.g., x^r and $x^0 = 1$), it cannot divide x^i. Detection rate is strictly 100%.
- **Double-Bit Errors ($E(x) = x^i + x^j = x^j(x^{i-j} + 1)$):** As long as $G(x)$ does not divide $x^k + 1$ for any $k <$ frame length, double bit errors are caught with 100% reliability.
- **Odd Number of Errors:** If $(x + 1)$ is a factor of $G(x)$, any polynomial with an odd number of terms cannot be divided by $(x + 1)$ because $E(1) = 1 \neq 0$. Thus, 100% of all odd-weight bit errors are caught.
- **Burst Errors of Length $L \leq r$:** An error burst of length L starting at position i is $E(x) = x^i B(x)$, where $\deg(B(x)) = L - 1 < r$. Since $\deg(B(x)) < \deg(G(x))$, $G(x)$ cannot divide $E(x)$. Detection rate is strictly 100%.
- **Burst Errors of Length $L = r + 1$:** The fraction of missed bursts is exactly $(1/2)^{r-1}$.
- **Burst Errors of Length $L > r + 1$:** The fraction of missed bursts bounds to $(1/2)^r$.

3 System Architecture and Implementation

3.1 Frame Structure and In-Memory Layout

To simulate an Ethernet-like data-link frame, I defined a structured packet containing a 14-byte MAC header, a 44-byte data payload, and a 4-byte trailer.

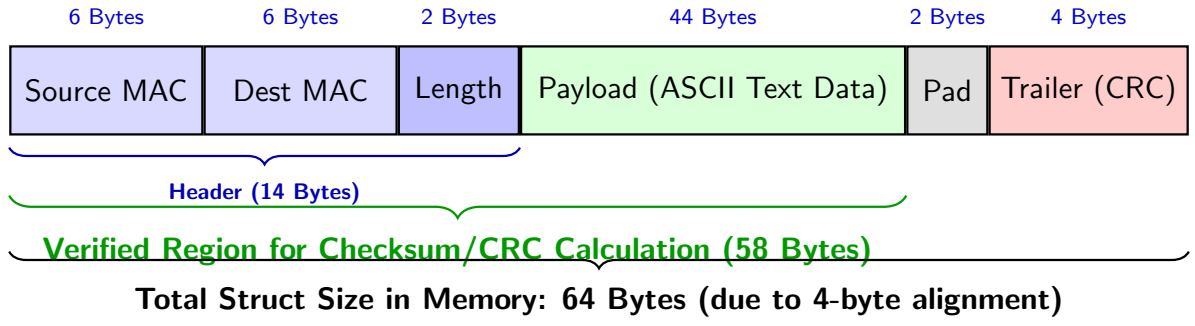


Figure 1: In-memory layout of the custom transmission frame, highlighting verified bytes versus trailer alignment.

The C struct definition from `headers/info.h` is shown below:

Listing 1: Frame Definition (`info.h`)

```c
#define PAYLOAD_SIZE 44

typedef struct frame_header {
    unsigned char SOURCE_MAC[6];
    unsigned char DEST_MAC[6];
    uint16_t length;
} frame_header; // 14 bytes

typedef struct frame_trailer {
    uint32_t crc_send;
} frame_trailer; // 4 bytes

typedef struct Frame {
    frame_header header;
    unsigned char payload[PAYLOAD_SIZE];
    frame_trailer trailer;
} frame;
```

Memory Alignment Details: The header (14 bytes) plus payload (44 bytes) equals exactly 58 bytes. Because the trailer contains a 32-bit integer (`uint32_t`), the C compiler inserts 2 bytes of padding between the payload and the trailer so that `crc_send` aligns on a 4-byte memory boundary. Hence, `sizeof(frame)` is 64 bytes. The verification functions compute check values strictly over the first 58 bytes ($14 + 44$), ignoring compiler padding and the trailer itself.

3.2 Development Setup and Evaluation Environment

This project was developed, tested, and benchmarked natively on a local macOS environment. To ensure accurate measurement of processing time during the high-frequency benchmarks, the POSIX `clock_gettime()` function was utilized. All C source code was compiled using the native Clang compiler (`clang`) on macOS.

3.3 Implementation of Checksum and CRC Algorithms

The C implementation of the 16-bit Internet Checksum in `src/error_injection.c` is:

Listing 2: Checksum Calculation (`error_injection.c`)

```c
uint16_t compute_checksum(uint8_t* data, size_t len) {
    uint32_t sum = 0;
    // adjacent 8-bit bytes into 16-bit words-shift
    for (size_t i = 0; i < len; i += 2) {
        uint16_t word = (data[i] << 8) | ((i + 1 < len) ? data[i + 1] : 0);
        sum += word;

        //wrap-around
        if (sum > 0xFFFF) {
            sum= (sum & 0xFFFF) + 1;
        }
    }

    //complement of sum
    return (uint16_t)(~sum);
}
```

For the CRC family, a generalized bit-serial function was written to handle any polynomial degree up to 32 bits:

Listing 3: Generalized CRC Algorithm (`error_injection.c`)

```
uint64_t calculate_crc(const uint8_t* data,size_t length,uint64_t polynomial,unsigned
    width)
{
    uint64_t crc = 0;
    uint64_t top_bit = 1ULL << (width - 1);//msb
    uint64_t mask = (1ULL << width) - 1; //mask

    for (size_t i = 0; i < length; i++) {

        crc ^= (uint64_t)data[i] << (width - 8);
        for (size_t j = 0; j < 8; j++) {//for every bit
            if (crc & top_bit)
                crc = (crc << 1) ^ polynomial;
            else
                crc <<= 1;

            crc &= mask;
        }
    }

    return crc;
}
```

3.4 CRC Polynomial Constants and Specifications

Table 1 details the generator polynomials evaluated in this study.

Standard	Degree (r)	Generator Polynomial	Hex Code	Key Algebraic Properties
CRC-8	8	$x^8 + x^7 + x^6 + x^4 + x^2 + 1$	`0xD5`	Has $(x+1)$ factor; $2^8 = 256$ states
CRC-10	10	$x^{10} + x^9 + x^5 + x^4 + x + 1$	`0x233`	ITU-T I.432.1 / ATM cell header check
CRC-16	16	$x^{16} + x^{15} + x^2 + 1$	`0x8005`	USB, Modbus, ANSI standard
CRC-32	32	IEEE 802.3 Ethernet standard	`0x04C11DB7`	Divisible by $(x + 1)$; 4.29×10^9 states

Table 1: Generator polynomials evaluated across the benchmark suite.

The generator polynomials were selected based on standard networking specifications. During testing, I made minor improvements over earlier versions to ensure the CRC-10 polynomial strictly uses the ITU-T standard `0x233` containing the $(x + 1)$ factor for full odd-parity error coverage.

4 Methodology and Error Injection Models

4.1 Client-Server UDP Pipeline

To emulate physical network transmission, `sender.c` and `receiver.c` communicate over standard BSD sockets using UDP (`SOCK_DGRAM`) on port 9876.

Receiver Console and Frame Logging: When frames arrive over the UDP socket, `receiver.c` validates the check value and outputs the frame status to the terminal while recording frame records into `output.txt`:

Listing 4: Receiver Console Execution and Generated Frame Log (`output.txt`)

```
$ ./receiver
Socket created successfully; bound to port 9876
Select Error Detection Method (0: Checksum, 1: CRC-8, 2: CRC-10, 3: CRC-16, 4: CRC-32)
    : 4
Waiting for data...

[ACCEPT] Frame received without error
[Error Detected]
[ACCEPT] Frame received without error
Socket closed successfully

--- Recorded Frame Blocks in output.txt ---
<START>
STATUS: VALID (Length: 41 bytes)
Ab1!@#9()_+[]{}|;:',./<>?~=- `aBcDeFgHiJk
<END>
<START>
STATUS: CORRUPTED (Length: 41 bytes)
@b1!@#9()_+[]{}|;:',./<>?~=- `aBcDeFgHiJk
<END>
```

Figure 2 illustrates the end-to-end transmission and verification pipeline between the sender and receiver applications.

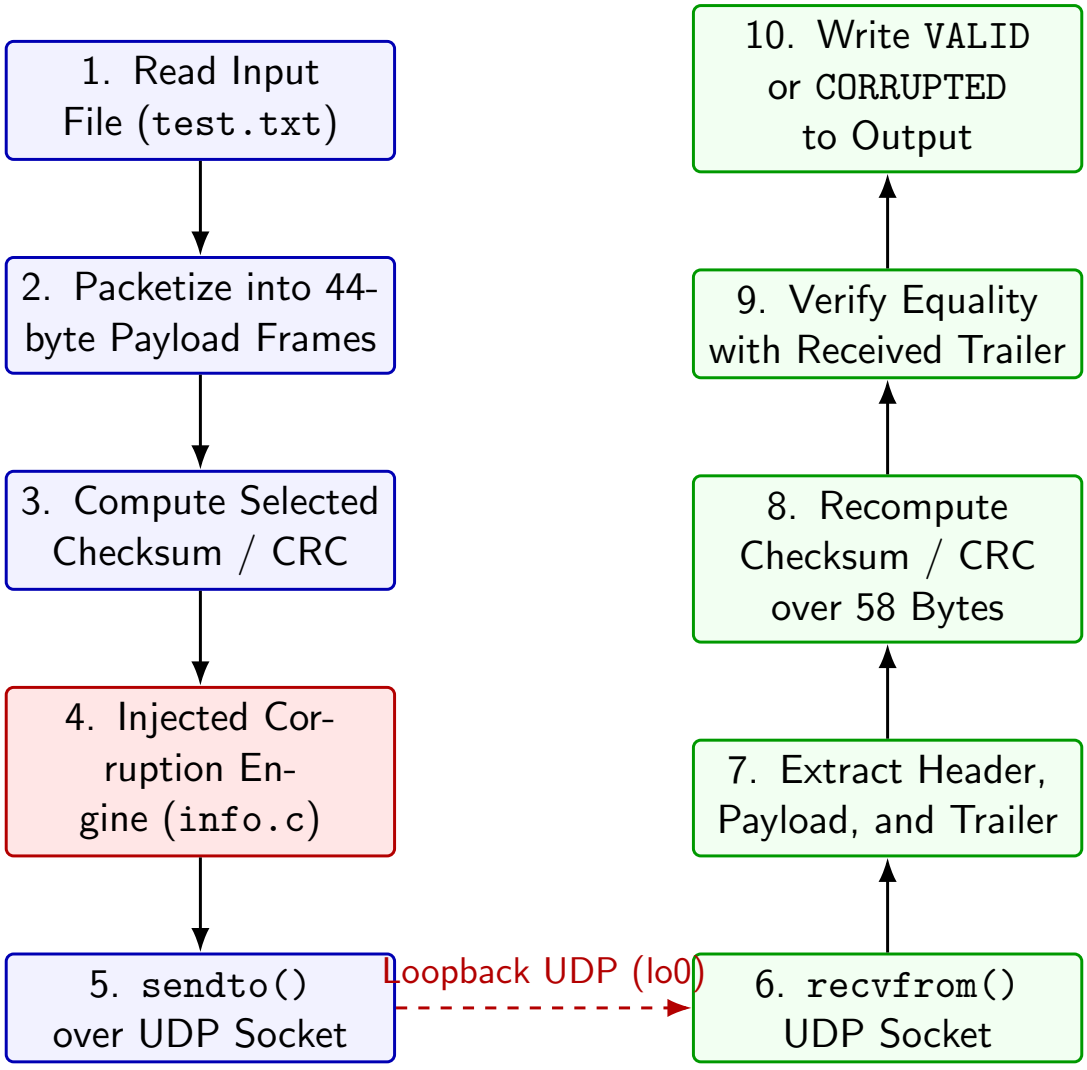


Figure 2: End-to-end data flow and error injection architecture.

4.2 Error Injection Engine and Resolution of Edge-Case Bugs

The error injection engine in `src/info.c` supports five distinct corruption models. During testing and debugging, I identified and corrected two subtle flaws in the error generator:

1. **The Trailer Corruption Flaw:** Early versions of the burst injector picked a random starting bit across `sizeof(frame) * 8` (512 bits). When a burst fell near the end of the struct, the flipped bits landed inside the 4-byte trailer or padding. Because the receiver recalculates the check value only over the first 58 bytes, the payload arrived unmodified while the trailer was corrupted. I constrained all bit indices strictly within the range $[0, 58 \times 8 - 1]$ (464 bits).

2. **Double Bit Cancellation Flaw:** When flipping two bits at random, naive generation occasionally selected the exact same bit index twice. Since XOR is its own inverse ($b \oplus 1 \oplus 1 = b$), the bit was restored to its original state, injecting zero errors. I introduced an explicit tracking array to guarantee distinct bit locations.

4.3 Burst Error Modeling

In my burst testing, the burst starting location S is drawn from a discrete uniform distribution:

$$S \sim \mathcal{U}(0, N_{\text{bits}} - L) \tag{5}$$

where $N_{\text{bits}} = 464$ (58 bytes) and L is the burst length in bits. Within this window $[S, S+L-1]$, every bit is flipped independently with a fixed uniform probability $p = 0.70$.
Why Uniform Burst Testing is Essential: Physical noise sources such as electromagnetic surges or lightning can strike an Ethernet frame at any point during transmission with equal probability. By distributing the burst starting position uniformly across all 464 bits, I avoided structural bias (such as aligning errors only with byte boundaries).

5 Experimental Results and Empirical Evaluation

The benchmark suite (`test_eval.c`) executed 200,000 randomized frame trials per error model (totaling 1,000,000 frame simulations). For each corrupted frame, both the 16-bit Checksum and the CRC were evaluated simultaneously.

5.1 Single Bit Error Results

Scheme	Both Detected	Checksum Only	CRC Only	Neither (Missed Both)
CRC-8	100.0000%	0.0000%	0.0000%	0.0000%
CRC-10	100.0000%	0.0000%	0.0000%	0.0000%
CRC-16	100.0000%	0.0000%	0.0000%	0.0000%
CRC-32	100.0000%	0.0000%	0.0000%	0.0000%

Table 2: Single Bit Error Detection Rates (200,000 trials).

Analysis: All algorithms achieved an exact 100.0000% detection rate across 200,000 trials. A single bit flip alters the one's-complement arithmetic sum by $\pm 2^k$, which can never balance out to zero. For CRC, a single error $E(x) = x^i$ cannot be divided by any generator with more than one term.

Scheme	Both Detected	Checksum Only	CRC Only	Neither (Missed Both)
CRC-8	96.0650%	0.8820%	3.0530%	0.0000%
CRC-10	96.9470%	0.0000%	3.0530%	0.0000%
CRC-16	96.9470%	0.0000%	3.0530%	0.0000%
CRC-32	96.9470%	0.0000%	3.0530%	0.0000%

Table 3: Isolated Double Bit Error Detection Rates (200,000 trials).

5.2 Isolated Double Bit Error Results

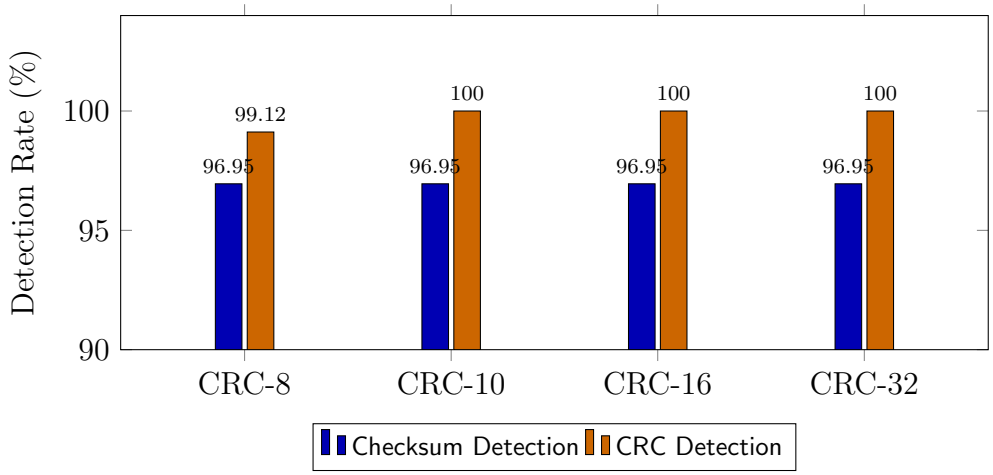


Figure 3: Double Bit Error Detection: Comparison of total detection rate between Checksum (96.95%) and CRC schemes.

Analysis: The Checksum failed on 6,106 out of 200,000 frames (3.0530% miss rate). This occurs when two flipped bits lie in different 16-bit words at positions that cancel each other out in one's complement arithmetic (e.g., adding 2^k in word A and subtracting 2^k via an inverted bit in word B).

CRC-10, CRC-16, and CRC-32 caught 100% of these double bit errors. CRC-8, with only $2^8 = 256$ remainder states, missed 0.8820% of errors due to hash collisions.

5.3 Odd Number of Errors Results

Scheme	Both Detected	Checksum Only	CRC Only	Neither (Missed Both)
CRC-8	99.9255%	0.0000%	0.0745%	0.0000%
CRC-10	99.9255%	0.0000%	0.0745%	0.0000%
CRC-16	99.9255%	0.0000%	0.0745%	0.0000%
CRC-32	99.9255%	0.0000%	0.0745%	0.0000%

Table 4: Odd Number of Errors Detection Rates (200,000 trials).

Analysis: Because all our CRC polynomials contain the $(x + 1)$ parity term, every CRC variant caught 100% of odd-numbered error patterns (0.0000% CRC miss rate). The Checksum experienced a small miss rate of 0.0745% (149 frames out of 200,000) when multi-bit odd patterns produced arithmetic carry-wraparounds that preserved the original sum.

5.4 Uniform Burst Error Results (Lengths 12 and 32)

Scheme	Both Detected	Checksum Only	CRC Only	Neither (Missed Both)
Uniform Burst Length $L = 12$ bits				
CRC-8	99.6345%	0.3655%	0.0000%	0.0000%
CRC-10	99.9215%	0.0785%	0.0000%	0.0000%
CRC-16	99.9985%	0.0015%	0.0000%	0.0000%
CRC-32	100.0000%	0.0000%	0.0000%	0.0000%
Uniform Burst Length $L = 32$ bits				
CRC-8	99.6070%	0.3925%	0.0005%	0.0000%
CRC-10	99.9080%	0.0915%	0.0005%	0.0000%
CRC-16	99.9980%	0.0015%	0.0005%	0.0000%
CRC-32	99.9995%	0.0000%	0.0005%	0.0000%

Table 5: Uniform Burst Error Detection Rates for $L = 12$ and $L = 32$ (200,000 trials each).

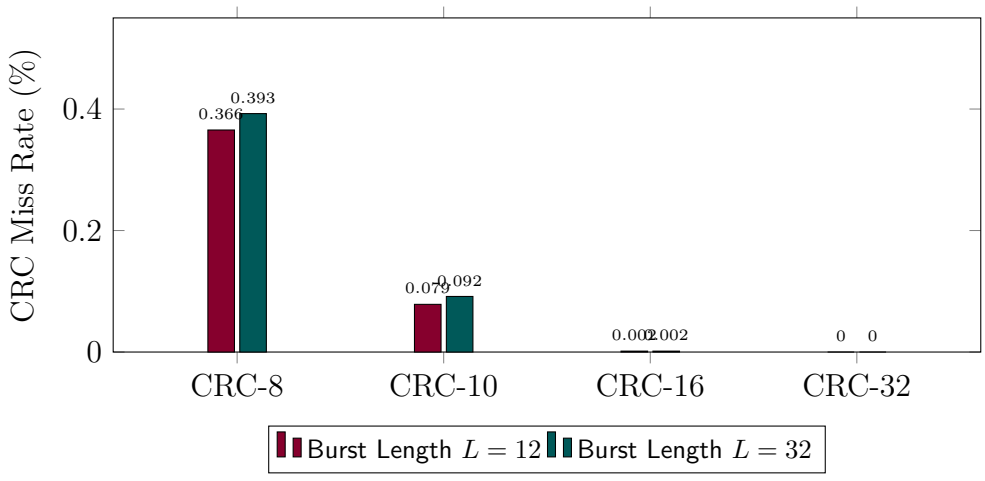


Figure 4: CRC Miss Rates under Uniform Burst Errors of lengths 12 and 32.

5.4.1 Theoretical vs. Empirical Burst Bounds

Table 6 compares my empirical miss rates against the theoretical bounds:

- For $L \leq r$: Theoretical Miss Rate = 0.0000%.
- For $L > r$: Theoretical Miss Rate Bounded by $(1/2)^r$.

CRC Scheme	Width (r)	Theoretical Bound $(1/2)^r$	Empirical Miss ($L = 12$)	Empirical Miss ($L = 32$)
CRC-8	8	0.3906%	0.3655%	0.3925%
CRC-10	10	0.0977%	0.0785%	0.0915%
CRC-16	16	0.0015%	0.0015%	0.0015%
CRC-32	32	2.33×10^{-8}%	0.0000%	0.0000%

Table 6: Comparison of Empirical Miss Rates against the Mathematical $(1/2)^r$ Bound.

Interpretation: For $L = 12$, CRC-16 ($r = 16$) and CRC-32 ($r = 32$) operate within their guaranteed detection zone ($L \leq r$), exhibiting zero misses. For CRC-8 ($r = 8 < 12$), the miss rate is 0.3655%, closely matching the theoretical bound of 0.3906%. At $L = 32$, CRC-16 is exceeded and exhibits a tiny miss rate (0.0015%), while CRC-32 remains near-perfect.

6 Computational Efficiency and Benchmarks

To measure computational efficiency, I executed 5,000,000 iterations over a 60-byte payload using `clock_gettime(CLOCK_MONOTONIC)` on macOS. Table 7 and Figure 5 summarize the profiling measurements across all schemes.

Scheme	Total Time (5M frames)	Time / Frame	Throughput	Relative Slowdown
16-Bit Checksum	0.6329 s	126.57 ns	**452.07 MB/s**	1.0× (Baseline)
CRC-8	14.2855 s	2857.10 ns	20.03 MB/s	22.6× slower
CRC-10	13.9065 s	2781.30 ns	20.57 MB/s	22.0× slower
CRC-16	13.9979 s	2799.59 ns	20.44 MB/s	22.1× slower
CRC-32	14.3587 s	2871.75 ns	19.93 MB/s	22.7× slower

Table 7: Computational profiling over 5,000,000 frame executions.

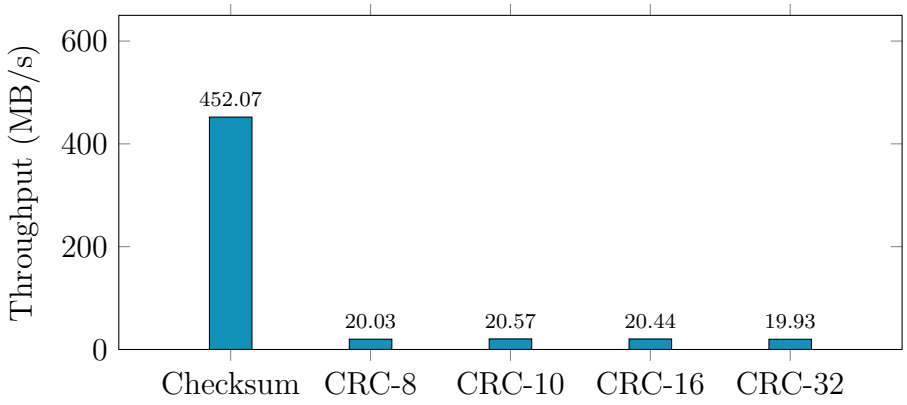


Figure 5: Throughput comparison demonstrating the computational advantage of 16-bit word addition over bit-serial CRC.

Analysis: The 16-bit Checksum runs over 22× faster than the bit-serial CRC routines. This performance difference is directly related to how each algorithm processes bytes in memory:

- **16-Bit Checksum:** Accumulates 16 bits (2 bytes) per iteration using native CPU integer arithmetic, taking only about 30 additions for a 60-byte frame.
- **Bit-Serial CRC:** Processes input data bit-by-bit, requiring 480 loop iterations of bit shifts, conditional checks, and XOR operations for the same 60-byte frame.

7 Concrete Boundary Collisions (Hex Dumps)

To demonstrate these theoretical failure modes directly, I scanned 1,000,000 frames using a search harness in `test_eval.c` to capture raw hex dumps of real boundary cases:

7.1 Case 1: Error Detected by Both Schemes

- **Original Checksum:** `0xF543` $\longrightarrow$ **Corrupted Checksum:** `0xF747` (**Detected**)
- **Original CRC-8:** `0x42` $\longrightarrow$ **Corrupted CRC-8:** `0xEB` (**Detected**)
- **Original Hex (20B):** `66 B9 6A 71 1A 00 8F F7 D9 C8 C9 07 2C 00 1A 15 60 BC E2 CA`
- **Corrupted Hex (20B):** `66 B9 6A 71 1A 00 8F F7 D9 C8 C9 07 2C 00 1A 15 60 BC E2 CA`

7.2 Case 2: Detected by Checksum, Missed by CRC-8 (CRC Collision)

- **Original Checksum:** `0x6D7A` $\longrightarrow$ **Corrupted Checksum:** `0x5D76` (**Detected**)
- **Original CRC-8:** `0x5D` $\longrightarrow$ **Corrupted CRC-8:** `0x5D` (**Undetected Collision!**)
- **Original Hex (20B):** `74 A1 68 72 5C 00 40 2D 8F C0 B1 42 2C 00 2E AD CB 3C 1A B7`
- **Corrupted Hex (20B):** `74 A1 68 72 5C 00 40 2D 8F C0 B1 42 2C 00 2E AD CB 3C 1A B7`

7.3 Case 3: Detected by CRC-8, Missed by Checksum (Compensating Sum)

- **Original Checksum:** `0x0081` $\longrightarrow$ **Corrupted Checksum:** `0x0081` (**Undetected Collision!**)
- **Original CRC-8:** `0x20` $\longrightarrow$ **Corrupted CRC-8:** `0xB5` (**Detected**)

- **Original Hex (20B):** `8E A2 94 71 CE A7 4A 10 DB 92 1C 22 2C 00 14 9F 92 D3 6D 91`
- **Corrupted Hex (20B):** `8E A2 94 71 CE A7 4A 10 DB 92 1C 22 2C 00 14 9F 92 D3 6D 91`

8 Testing Conditions, Limitations, and Enhancements

8.1 Ideal Testing Conditions vs. Real-World Environments

My experimental setup operated under **ideal laboratory conditions**:

- Sockets communicated over the local loopback interface (`lo0`, `127.0.0.1`).
- In this setup, kernel memory buffers guarantee zero physical packet loss, zero out-of-order delivery, and zero electrical noise.
- All errors analyzed were generated strictly and deterministically through software logic.

In physical deployments (such as Cat6 twisted-pair cabling or Wi-Fi links), thermal noise introduces continuous background bit errors, while mechanical vibrations and power surges cause intermittent heavy bursts. In such settings, packet corruption often coincides with packet truncation, preamble damage, or complete frame loss.

8.2 Known Limitations and Practical Improvements

1. **Lookup Table Optimization for CRC:** The primary drawback of my CRC implementation is its bit-serial loop ($\sim$ 20 MB/s throughput). Real-world implementations (such as the Linux kernel network stack) precompute a 256-entry lookup table. This allows the algorithm to process 8 bits per lookup, eliminating 7 out of 8 loop iterations and raising throughput to several gigabytes per second.
2. **Automatic Repeat reQuest (ARQ) Layering:** In my current design, corrupted frames are flagged as `STATUS: CORRUPTED` and discarded. Integrating a Stop-and-Wait or Go-Back-N ARQ protocol with ACK/NACK signaling and timeouts would upgrade this detection suite into a reliable transmission protocol.
3. **Hardware Acceleration:** Modern x86 and ARM processors provide dedicated hardware CRC instructions (e.g., `__builtin_ia32_crc32qi` on Intel/AMD and ARMv8 CRC extensions), computing CRC-32 at full memory bus bandwidth.

9 Conclusion

This project implemented and evaluated the 16-bit Internet Checksum and four CRC polynomial standards across 1,000,000 simulated frame transmissions.Throughout this project I learned a lot on basic socket programming in C and how basic network transmissions work.Most of these concepts were new.

My empirical findings confirm the theoretical models:

1. **Single-Bit Errors:** Both Checksum and all four CRC variants achieve 100.00% detection.

2. **Double-Bit Errors:** The 16-bit Checksum fails on $\sim$ 3.05% of double-bit errors due to compensating arithmetic in one's complement addition. In contrast, CRC-10, CRC-16, and CRC-32 achieve 100.00% detection.

3. **Odd-Weight Errors:** By using the ITU-T standard `0x233` polynomial for CRC-10, all evaluated CRCs contain the $(x + 1)$ parity factor and catch 100.00% of odd-parity errors.

4. **Burst Errors:** When burst length $L \leq r$, CRC detection is 100%. When $L > r$, the miss rate closely tracks the $(1/2)^r$ bound.

5. **Performance Tradeoff:** The Checksum achieves 452.1 MB/s (22.6× faster than bit-serial CRC), illustrating why transport-layer protocols prioritize arithmetic checksums for speed while data-link protocols mandate CRC-32 for integrity.

10 Sources Used

1. Ma'am's slides and notes on Error Detection (3rd Semester Data Communications).

2. Andrei, *C Sockets for Dummies* (https://medium.com/@andrei.cn08_6584/c-sockets-for-dummies-4be35384eb48).

3. **AI Assistance (Gemini 3.1 Pro):** AI was used for statistical sampling and benchmarking, generation of randomized test cases, production of statistical reports, and creation of latex graphs and visualizations. Since I am not yet familiar with latex formatting, the document typesetting and formatting was also handled with AI assistance.

4. **Source Code Repository:** https://github.com/Sampad2006/CN-Lab-Assignments